HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

KHOA CÔNG NGHỆ THÔNG TIN 1

BÁO CÁO
AN TOÀN VÀ BẢO MẬT HỆ THỐNG THÔNG TIN

Đề tài: Giải thuật tạo chữ ký số sử dụng RSA

Giảng viên hướng dẫn: TS. Bùi Văn Công

Nhóm 1

1. Hoàng Văn Thiện B23DCKH114
2. Nguyễn Ngọc Phúc B23DCKH090
3. Nguyễn Quang Trung B23DCKH122
4. Đỗ Quốc Tấn B23DCKH104

Hà Nội, 04-2026

PHÂN CÔNG

Thành viên	Mã sinh viên	Vai trò
Hoàng Văn Thiện	B23DCKH114	Nội dung, báo cáo
Nguyễn Ngọc Phúc	B23DCKH090	Thiết kế Slide
Nguyễn Quang Trung	B23DCKH122	Nội dung, báo cáo, cài đặt
Đỗ Quốc Tấn	B23DCKH104	Thuyết trình

MỤC LỤC

1. Giới thiệu 4
2. Một số khái niệm 5
2.1. Khái niệm chữ ký số 5
2.2. Khái niệm hàm băm 6
2.3. Hệ mật mã hóa công khai 8
3. Nội dung 9
3.1. Kiến trúc 9
3.1.1. Kiến trúc chữ ký số tổng quát 9
3.1.2. Kiến trúc chữ ký số RSA 10
3.2. Giải thuật 11
3.2.1. Giải thuật RSA được dùng trong việc tạo khóa 11
3.2.2. Các bước hoạt động 11
3.3. Các điểm yếu của chữ ký số sử dụng giải thuật RSA 14
3.3.1. Chữ ký số nói chung 14
3.3.2. Chữ ký số sử dụng RSA 16
3.3.3. Các cuộc tấn công bảo mật hiện đại 17
3.4. Ứng dụng thực tế của chữ ký số RSA 19
3.5. Cài đặt giải thuật sử dụng Python 22
4. Kết luận 26
5. Tài liệu tham khảo 27

1. Giới thiệu

Trong bối cảnh chuyển đổi số toàn cầu và sự gia tăng mạnh mẽ của các hệ thống phân tán, việc đảm bảo an toàn thông tin trong môi trường truyền thông mở đã trở thành một yêu cầu mang tính nền tảng đối với mọi hạ tầng công nghệ. Các mối đe dọa như giả mạo danh tính, thay đổi dữ liệu trái phép và chối bỏ giao dịch đặt ra nhu cầu cấp thiết đối với các cơ chế bảo mật có tính xác minh mạnh mẽ.

Sự ra đời của mật mã khóa công khai bởi Whitfield Diffie và Martin Hellman vào năm 1976 đã mở ra một hướng tiếp cận hoàn toàn mới, loại bỏ sự phụ thuộc vào việc trao đổi khóa bí mật trước đó. Tiếp nối nền tảng này, thuật toán RSA được giới thiệu lần đầu vào năm 1977 ba nhà khoa học Ron Rivest, Adi Shamir và Leonard Adleman đánh dấu bước ngoặt trong lịch sử mật mã học hiện đại.

Không chỉ dừng lại ở chức năng mã hóa, RSA còn cung cấp một cơ chế nền tảng cho chữ ký số – một thành phần không thể thiếu trong các hệ thống giao dịch điện tử, tài chính số và hạ tầng khóa công khai (PKI). Chữ ký số dựa trên RSA đảm bảo ba thuộc tính an ninh cốt lõi:

- Tính xác thực (Authenticity): Xác minh danh tính thực thể thông qua khóa công khai
- Tính toàn vẹn (Integrity): Phát hiện mọi thay đổi trái phép trên dữ liệu
- Tính chống chối bỏ (Non-repudiation): Cung cấp bằng chứng pháp lý không thể phủ nhận

2. Một số khái niệm

2.1. Khái niệm chữ ký số

Chữ ký số (Digital Signature) là một chuỗi dữ liệu liên kết với một thông điệp (message) và thực thể tạo ra thông điệp, có vai trò tương tự chữ ký tay trong văn bản giấy, giúp xác nhận người gửi và đảm bảo nội dung không bị thay đổi.

Giải thuật kiểm tra chữ ký số (Digital Signature Verification Algorithm) là một phương pháp xác minh tính xác thực của chữ ký số, nghĩa là kiểm tra xem chữ ký đó có thực sự được tạo ra bởi đúng người gửi hay không.

Một hệ chữ ký số (Digital Signature Scheme) bao gồm:

- Giải thuật tạo chữ ký số.
- Giải thuật kiểm tra chữ ký số.
- Quá trình tạo chữ ký số (Digital Signature Signing Process) bao gồm:
    - Giải thuật tạo chữ ký số.
    - Phương pháp chuyển dữ liệu thông điệp thành dạng có thể ký được.
- Quá trình kiểm tra chữ ký số (Digital Signature Verification Process) bao gồm:
    - Giải thuật kiểm tra chữ ký số.
    - Phương pháp khôi phục dữ liệu từ thông điệp.

Chữ ký số thường được sử dụng trong:

- Hợp đồng điện tử.
- Giao dịch ngân hàng.
- Khai báo thuế.
- Hóa đơn điện tử.
- Email bảo mật.

2.2. Khái niệm hàm băm

Hàm băm là một hàm toán học dùng để chuyển đổi một thông điệp có độ dài bất kỳ thành một dãy bit có độ dài cố định.

Dãy bit này được gọi là:

- Thông điệp rút gọn (Message Digest).
- Giá trị băm (Hash Value).

Giá trị băm đại diện cho nội dung của thông điệp ban đầu.

Đặc điểm của hàm băm:

- Cùng một thông điệp luôn cho ra cùng một giá trị băm.
- Chỉ cần thay đổi một ký tự nhỏ trong thông điệp thì giá trị băm sẽ thay đổi hoàn toàn.
- Không thể khôi phục lại thông điệp ban đầu chỉ từ giá trị băm.
- Hàm băm giúp kiểm tra tính toàn vẹn của dữ liệu.
- Hàm băm giúp giảm kích thước dữ liệu cần ký vì thay vì ký toàn bộ văn bản, hệ thống chỉ cần ký lên giá trị băm.
- Hàm băm giúp tăng tốc độ xử lý của chữ ký số.

Các thuật toán băm phổ biến

a) Hàm băm SHA-1

- SHA-1 là một thuật toán băm nhận đầu vào có độ dài nhỏ hơn 2^64 bit.
- SHA-1 xử lý dữ liệu theo từng khối 512 bit.
- Sử dụng 5 thanh ghi, mỗi thanh ghi dài 32 bit.
- Kết quả cuối cùng là một chuỗi băm dài 160 bit.

b) Hàm băm MD5

- MD5 là một thuật toán băm được phát triển năm 1991 bởi Ronald Rivest.
- Nhận đầu vào có độ dài bất kỳ.
- Xử lý dữ liệu theo từng khối 512 bit.
- Sử dụng 4 thanh ghi, mỗi thanh ghi dài 32 bit.
- Kết quả cuối cùng là một chuỗi băm dài 128 bit.

c) Hàm băm SHA-2

- SHA-2 là họ các thuật toán băm do NIST công bố năm 2001.
- Bao gồm nhiều biến thể: SHA-224, SHA-256, SHA-384, SHA-512.
- Nhận đầu vào có độ dài nhỏ hơn 2128 bit (đối với SHA-512).
- Xử lý dữ liệu theo từng khối 512 bit (SHA-256) hoặc 1024 bit (SHA-512).
- Sử dụng số lượng thanh ghi khác nhau tùy biến thể:
  - SHA-256: 8 thanh ghi, mỗi thanh ghi dài 32 bit.
  - SHA-512: 8 thanh ghi, mỗi thanh ghi dài 64 bit.
- Kết quả cuối cùng là chuỗi băm có độ dài 224, 256, 384 hoặc 512 bit tùy biến thể

d) Hàm băm SHA-3

- SHA-3 được chuẩn hóa năm 2015, dựa trên thuật toán Keccak.
- Nhận đầu vào có độ dài bất kỳ.
- Xử lý dữ liệu theo cơ chế "sponge construction" (hấp thụ và nén dữ liệu).
- Không dùng cấu trúc Merkle–Damgård như SHA-1 và SHA-2.
- Độ dài đầu ra linh hoạt: 224, 256, 384, 512 bit.
- Đặc điểm nổi bật: chống lại nhiều kiểu tấn công toán học mới, thiết kế khác biệt so với SHA-2.

	MD5	SHA-1	SHA-2 (SHA-256/512)	SHA-3 (Keccak)
Năm ra đời	1991	1995	2001	2015
Độ dài đầu ra	128 bit	160 bit	224–512 bit	224–512 bit
Kích thước khối xử lý	512 bit	512 bit	512/1024 bit	Linh hoạt (sponge)
Số thanh ghi	4 × 32 bit	5 × 32 bit	8 × 32/64 bit	Phụ thuộc cấu trúc sponge
Mức độ bảo mật	Thấp (không an toàn)	Trung bình	Cao, chuẩn hiện tại	Cao, xu hướng tương lai
Ứng dụng	Checksum cũ	SSL/TLS cũ	Chữ ký số, HTTPS, Blockchain	IoT, Blockchain, hệ thống mới

Giả sử thông điệp : Message = "Hello World"

- MD5: fc3ff98e8c6a0d3087d515c0473f8677
- SHA-256: a591a6d40bf420404a011733cfb7b190d62c65bf0bcda32b57b277d9ad9f146e
- SHA3-256: 644bcc7e5646dee8f5e951b37fcb64c5e8d5a438f0e0f2e7c54b4da9c6b6d9f0

2.3. Hệ mật mã khóa công khai

Hệ chữ ký số hoạt động hoàn toàn dựa vào nền tảng của hệ mật mã bất đối xứng (hay mật mã khóa công khai).

RSA hoạt động dựa trên việc sử dụng hai khóa có mối quan hệ đặc biệt về mặt toán học:

- Khóa công khai (Public Key): Được công bố rộng rãi trên mạng cho mọi người.
- Khóa bí mật (Private Key): Được lưu giữ an toàn tuyệt đối và chỉ có chủ nhân duy nhất nắm quyền.

Ví dụ

Ta có thể mô phỏng trực quan một hệ mật mã khóa công khai như sau:

- Bob muốn gửi cho Alice một thông tin mật mà Bob muốn duy nhất Alice có thể đọc được.
- Alice gửi cho Bob một chiếc hộp có khóa đã mở sẵn.
- Chiếc hộp này đóng vai trò là khóa công khai.
- Bob cho thư vào hộp rồi khóa lại.
- Sau khi khóa, Bob cũng khlong thể mở lại được.
- Bob gửi chiếc hộp cho Alice.
- Alice dùng chìa khóa riêng của mình để mở hộp và đọc nội dung.

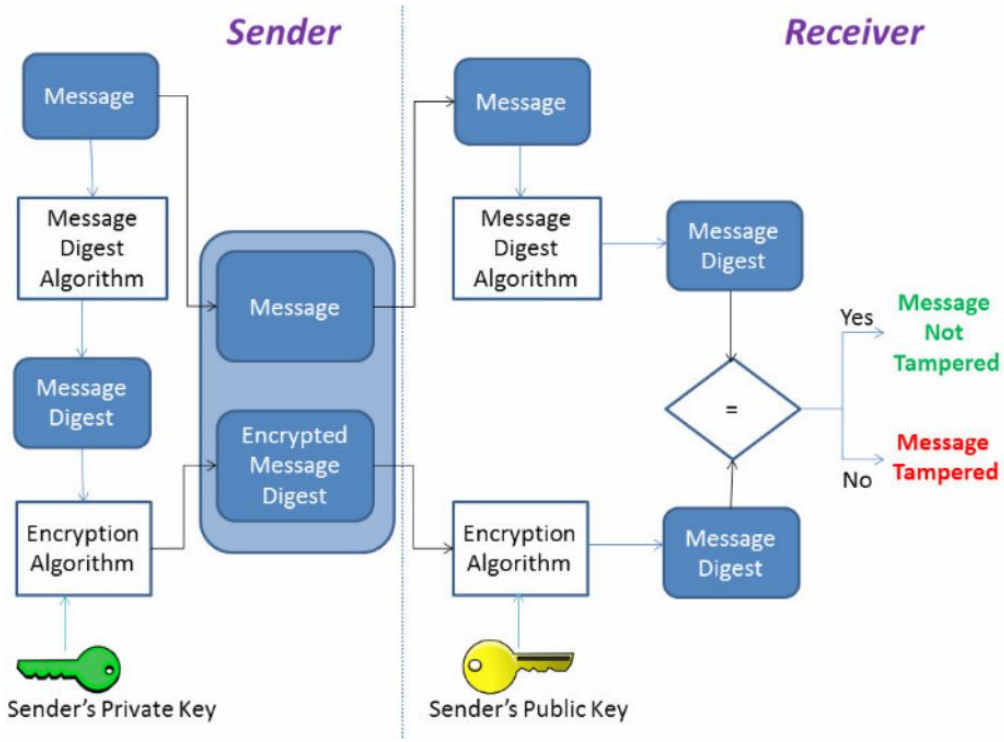
Trong ví dụ này:

- Chiếc hộp khóa mở = khóa công khai.
- Chìa khóa của Alice = khóa bí mật

3. Nội dung

3.1 Kiến trúc

3.1.1. Kiến trúc chữ ký số tổng quát



a) Quá trình ký (bên gửi)

- Tính toán chuỗi đại diện (Message Digest / Hash Value) của thông điệp bằng một giải thuật băm.
- Chuỗi đại diện được ký bằng khóa riêng (Private Key) của người gửi và giải thuật tạo chữ ký.
- Kết quả thu được là chữ ký số (Digital Signature), hay còn gọi là chuỗi đại diện đã được mã hóa.
- Thông điệp ban đầu được ghép với chữ ký số để tạo thành thông điệp đã ký (Signed Message).
- Thông điệp đã ký được gửi cho người nhận.

b) Quá trình kiểm tra chữ ký (bên nhận)

- Tách chữ ký số và thông điệp gốc ra khỏi thông điệp đã ký.
- Tính lại chuỗi đại diện MD1 của thông điệp gốc bằng cùng giải thuật băm đã dùng khi ký.
- Dùng khóa công khai (Public Key) của người gửi để giải mã chữ ký số và thu được chuỗi đại diện MD2.
- So sánh MD1 và MD2:
  - Nếu MD1 = MD2 thì chữ ký hợp lệ.
  - Nếu MD1 khác MD2 thì chữ ký không hợp lệ.
- Khi MD1 = MD2, ta có thể kết luận:
  - Thông điệp không bị thay đổi trong quá trình truyền.
  - Thông điệp thực sự do người gửi tạo ra.
  - Người gửi không thể phủ nhận việc đã gửi thông điệp.

3.1.2. Kiến trúc chữ ký số RSA

a. Quá trình ký (bên gửi)

- Tính toán chuỗi đại diện của thông điệp bằng giải thuật băm SHA-1.
- Chuỗi đại diện được ký bằng khóa riêng RSA của người gửi.
- Kết quả thu được là chữ ký số RSA.
- Ghép chữ ký số với thông điệp ban đầu.
- Gửi thông điệp đã ký cho người nhận.

b. Quá trình kiểm tra chữ ký (bên nhận)

- Tách chữ ký số RSA và thông điệp gốc.
- Tính lại giá trị băm MD1 bằng SHA-1.
- Dùng khóa công khai RSA của người gửi để giải mã chữ ký và thu được MD2.
- So sánh MD1 và MD2:
  - Nếu MD1 = MD2 thì chữ ký hợp lệ.
  - Nếu MD1 khác MD2 thì chữ ký không hợp lệ.

3.2. Giải thuật

3.2.1. Giải thuật RSA được dùng trong việc tạo khóa, mã hóa, giải mã

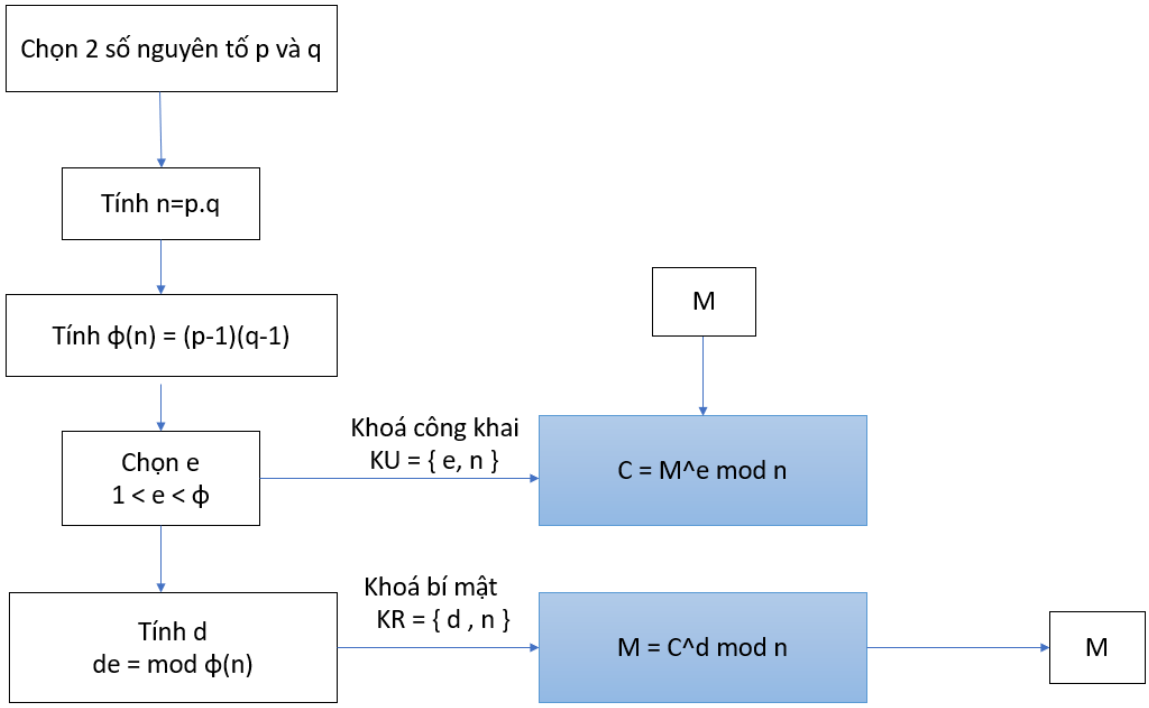
RSA là thuật toán mã hóa khóa công khai được xây dựng dựa trên tính khó của việc phân tích một số rất lớn thành tích của hai số nguyên tố.

RSA hoạt động dựa trên việc sử dụng hai khóa:

- Khóa công khai dùng để mã hóa hoặc kiểm tra chữ ký được công bố rộng rãi cho mọi
  người và được dùng để mã hóa
- Khóa bí mật dùng để giải mã hoặc tạo chữ ký. Những thông tin được mã hóa bằng khóa công khai chỉ có thể được giải mã bằng khóa bí mật tương ứng

3.2.2 Các bước hoạt động

Sơ đồ giải thuật:



Các bước tạo khóa RSA

- Bước 1: Chọn hai số nguyên tố lớn p và q.
- Bước 2: Tính:
  - $n = p \times q$
  - $\varphi(n) = (p - 1) \times (q - 1)$
- Bước 3: Chọn số e ($0 < e < \varphi(n)$) sao cho:
  - $\gcd(e, \varphi(n)) = 1$
- Bước 4: Tính d = e^-1 bằng cách dùng thuật toán Euclide :
  - Tìm số tự nhiên x sao cho $d = (x\varphi(n) + 1)/e$
- Bước 5: Khi đó:
  - Khóa công khai là (e, n).
  - Khóa bí mật là (d, n).

Các bước giải mã RSA

- Bước 1: Nhận bản mã C (Ciphertext) từ bên gửi.
- Bước 2: Sử dụng khóa bí mật (d,n) để tính:

  Trong đó:

  - Khóa công khai là (e, n).
  - Khóa bí mật là (d, n).
  - d là số mũ bí mật.
  - n là tích của hai số nguyên tố p và q.
- Bước 3: Thu được bản rõ M, chính là thông điệp gốc mà bên gửi đã mã hóa

Ví dụ minh họa tạo khóa và giải mã RSA

- Bước 1: Chọn hai số nguyên tố lớn
  - p=17,q=11
- Bước 2: Tính toán
  - $n = p \times q = 17 \times 11 = 187$
  - $\varphi(n) = (p - 1) \times (q - 1) = 16 \times 10 = 160$
  - Chọn e=7, vì gcd(7,160)=1.
- Bước 3: Chọn số e

- Chọn e=7, vì gcd(7,160)=1.

- Bước 4: Tính d
  - d = 23 (vì 23 x 7 = 161, và 161 mod(160) = 1)
- Bước 5: Khi đó:
  - Khóa công khai là (e = 7, n = 187).
  - Khóa bí mật là (d = 23, n = 187).

Quá trình mã hóa

Giả sử có một bản mã C được gửi đến, người nhận sẽ sử dụng khóa bí mật (23, 187) để giải mã theo công thức:

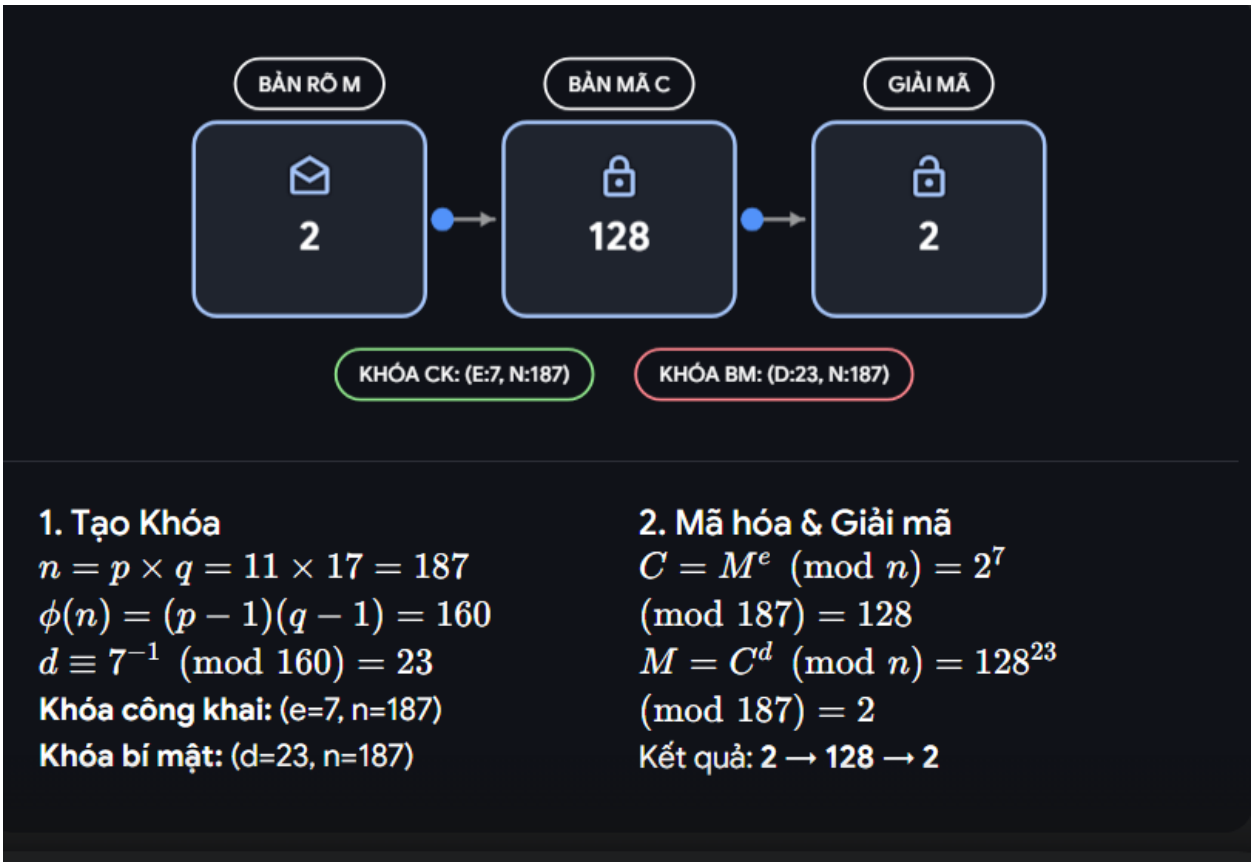
M = C^23 mod(187)

Nếu thông điệp ban đầu là M = 2

Quá trình mã hóa: C = 2^7 mod(187) = 128 (Bản mã C = 128)

Quá trình giải mã: M = 128^23 mod(187) = 2. (Khôi phục thành công bản rõ M = 2)

→ Kết quả khôi phục đúng thông điệp gốc M=2



3.3. Các điểm yếu của chữ ký số sử dụng giải thuật RSA

3.3.1. Chữ ký số nói chung

Sự xuất hiện của chữ ký số và chức năng tiền định của nó, đặc biệt là vai trò của nó như là một công cụ trong việc xác định tính nguyên gốc, xác định tác giả, bảo đảm tính toàn vẹn của tài liệu số, đã đóng một vai trò vô cùng quan trọng trong việc xác định địa vị pháp lý của tài liệu số trong giao dịch số.

Việc sử dụng chữ ký số trong phần lớn trường hợp là cơ sở khẳng định giá trị pháp lý của những văn bản điện tử tương đương với tài liệu giấy.

Hiện nay, chữ ký số là phương tiện duy nhất để xác nhận giá trị pháp lý của tài liệu điện tử.

Như vậy, với sự xuất hiện của chữ ký số, vấn đề giá trị pháp lý của tài liệu điện tử có thể coi như đã được giải quyết.

Việc sử dụng chữ ký số trong giao dịch cũng có những ưu điểm và bất cập nhất định. Dưới đây là những hạn chế của chữ ký số:

- Sự lệ thuộc vào máy móc và chương trình phần mềm: chữ ký số là một chương trình phần mềm máy tính. Để kiểm tra tính xác thực của chữ ký cần có hệ thống máy tính và phần mềm tương thích. Đây là hạn chế chung khi sử dụng văn bản điện tử và chữ ký số.

- Tính bảo mật không tuyệt đối: Nếu chữ ký bằng tay được thực hiện trên giấy, được ký trực tiếp và luôn đi kèm với vật mang tin, chữ ký tay không thể chuyển giao cho người khác, thì chữ ký số không như vậy.

Chữ ký số là một bộ mật mã được cấp cho người sử dụng, đây là phần mềm máy tính không phụ thuộc vào vật mang tin.

Chính vì vậy, trở ngại lớn nhất khi sử dụng chữ ký số là khả năng tách biệt khỏi chủ nhân của chữ ký.

Nói cách khác, chủ nhân của chữ ký số không phải là người duy nhất có được mật mã của chữ ký.

Tồn tại một số nhóm đối tượng có thể có được mật mã, đó là:

- Bộ phận cung cấp phần mềm.
- Bộ phận cài đặt phần mềm.
- Những người có thể sử dụng máy tính có cài đặt phần mềm.
- Hacker hoặc virus đánh cắp dữ liệu.

Ngoài ra, chủ nhân chữ ký số cũng có thể tự chuyển giao khóa bí mật của mình cho người khác.

Như vậy, tính bảo mật của chữ ký số không phải là tuyệt đối.

- Vấn đề bản gốc, bản chính: Nếu đối với tài liệu giấy, chữ ký được ký một lần và chỉ có một bản duy nhất được coi là bản gốc, thì với văn bản điện tử được ký bằng chữ ký số, người ta có thể sao chép vô hạn lần mà nội dung vẫn giống hoàn toàn với bản chính.

Bản copy từ bản chính và bản copy từ bản copy không có gì khác biệt so với bản chính duy nhất được ký.

Đây là một thách thức đối với công tác văn bản và cả nền hành chính.

Khái niệm bản gốc, bản chính trong văn bản hành chính sẽ phải xem xét lại đối với văn bản điện tử.

- Sự có thời hạn của chữ ký điện tử: Chữ ký điện tử là chương trình phần mềm được cấp có thời hạn cho người sử dụng.

Về lý thuyết, văn bản sẽ có hiệu lực pháp lý khi được ký trong thời hạn sử dụng của chữ ký.

Tuy nhiên, trên thực tế, hiệu lực pháp lý của văn bản hoàn toàn có thể bị nghi ngờ khi chữ ký số hết thời hạn sử dụng. Đây cũng là một hạn chế và thách thức rất lớn đối với việc sử dụng chữ ký số.

3.3.2. Chữ ký số sử dụng RSA

RSA chậm hơn nhiều so với các hệ mã đối xứng như DES hoặc AES.

Nguyên nhân là do RSA phải thực hiện các phép tính lũy thừa modulo với số rất lớn, trong khi AES và DES chỉ thực hiện các phép toán bit đơn giản nên nhanh hơn nhiều.

- Khi thực hiện bằng phần mềm, RSA chậm hơn DES khoảng 100 lần.
- Khi thực hiện bằng phần cứng, RSA chậm hơn DES khoảng 1000 lần.

a) Chi phí

RSA tốn nhiều chi phí tính toán do phải thực hiện:

- Sinh khóa.
- Mã hóa.
- Giải mã.
- Tính lũy thừa modulo với các số rất lớn.

Thông thường:

- Số mũ công khai e nhỏ hơn nhiều so với số mũ bí mật d.
- Vì vậy thời gian mã hóa thường nhanh hơn thời gian giải mã.

Quá trình mã hóa và giải mã tương đương với chi phí thực hiện các phép tính lũy thừa modulo n.

Để đảm bảo cho khóa bí mật được an toàn thì thường chọn số mũ công khai e nhỏ hơn nhiều so với số mũ bí mật d.

Do đó chi phí thời gian để thực hiện mã hóa dữ liệu nhỏ hơn nhiều so với thời gian giải mã.

b) Tốc độ của RSA

RSA chậm hơn từ 100 đến 1000 lần so với các hệ mã đối xứng.

Do đó RSA chủ yếu dùng để:

- Mã hóa khóa.
- Tạo chữ ký số.
- Xác thực.

RSA không phù hợp để mã hóa khối lượng dữ liệu lớn.

3.3.3. Các cuộc tấn công bảo mật hiện đại

- Độ dài khóa: Khóa RSA 1024-bit hiện nay được coi là không còn an toàn trước các siêu máy tính. Tiêu chuẩn bắt buộc hiện nay là 2048-bit hoặc cao hơn.
- Cơ chế Đệm (Padding): Để chống lại các cuộc tấn công toán học, RSA thực tế luôn sử dụng cơ chế đệm như PSS (Probabilistic Signature Scheme). Nếu không có đệm, hacker có thể suy diễn ra khóa từ các chữ ký tương tự nhau.
- Xu hướng ECC: Thuật toán ECDSA (Dựa trên đường cong Elliptic) đang dần thay thế RSA vì nó cung cấp độ bảo mật tương đương RSA 2048-bit nhưng với kích thước khóa chỉ 256-bit, giúp xử lý nhanh hơn rất nhiều trên điện thoại và thiết bị IoT.

Trong bối cảnh bảo mật hiện đại, bên cạnh RSA, các hệ mật mã dựa trên đường cong elliptic (Elliptic Curve Cryptography - ECC) ngày càng được sử dụng rộng rãi nhờ hiệu suất cao và độ an toàn tốt hơn trên cùng kích thước khóa.

Tiêu chí	RSA	ECC
Nguyên lý	Dựa trên bài toán phân tích số nguyên lớn	Dựa trên bài toán logarit rời rạc trên đường cong elliptic
Độ dài khóa	2048-bit	256-bit
Mức độ bảo mật	Cao	Cao hơn trên cùng kích thước khóa
Tốc độ xử lý	Chậm	Nhanh hơn
Tài nguyên sử dụng	Tốn nhiều CPU và bộ nhớ	Tiết kiệm tài nguyên
Ứng dụng	HTTPS, chữ ký số truyền thống	Mobile, IoT, Blockchain
Xu hướng	Đang dần thay thế	Xu hướng phát triển chính

Các kiểu tấn công phổ biến:

Timing Attack

Khái niệm: Hacker đo thời gian hệ thống thực hiện các phép toán mật mã. Nếu thời gian khác nhau tùy theo dữ liệu hoặc khóa, kẻ tấn công có thể suy ra thông tin về khóa bí mật.

Ví dụ: Trong RSA, thời gian tính toán phép lũy thừa modulo có thể tiết lộ số bit của khóa.

Phòng chống:

- Sử dụng thuật toán với thời gian thực thi hằng số (constant-time algorithms).
- Thêm nhiễu (random delays) để che giấu thời gian thực.

Chosen Ciphertext Attack (CCA)

Khái niệm: Hacker chủ động chọn bản mã (ciphertext) và yêu cầu hệ thống giải mã, từ đó suy ra thông tin về khóa hoặc bản rõ.

Ví dụ: RSA không có padding dễ bị tấn công CCA, vì kẻ tấn công có thể gửi bản mã đặc biệt để khai thác.

Phòng chống:

- Luôn sử dụng cơ chế đệm an toàn như RSA-OAEP hoặc RSA-PSS.
- Không cho phép người dùng tùy ý gửi bản mã để hệ thống giải mã.

Side-Channel Attack

Khái niệm: Khai thác thông tin rò rỉ ngoài luồng như điện năng tiêu thụ, bức xạ điện từ, âm thanh, hoặc nhiệt độ khi thiết bị thực hiện phép toán mật mã.

Ví dụ: Đo mức tiêu thụ điện năng của chip khi thực hiện RSA để suy ra khóa bí mật.

Phòng chống:

- Che giấu tín hiệu vật lý bằng cách thêm nhiễu.
- Thiết kế phần cứng chống rò rỉ (tamper-resistant hardware).
- Sử dụng kỹ thuật "masking" để làm ngẫu nhiên dữ liệu trong quá trình tính toán

3.4. Ứng dụng thực tế của chữ ký số RSA

Chữ ký số sử dụng RSA được ứng dụng rộng rãi trong nhiều lĩnh vực của đời sống và công nghệ thông tin, đóng vai trò quan trọng trong việc đảm bảo an toàn và tính pháp lý của dữ liệu.

a) Giao thức HTTPS (SSL/TLS)

RSA được sử dụng trong quá trình bắt tay (handshake) của giao thức HTTPS để:

- Xác thực máy chủ với trình duyệt
- Trao đổi khóa phiên an toàn

Điều này giúp bảo vệ dữ liệu truyền giữa người dùng và website khỏi bị nghe lén hoặc giả mạo.

b) Email bảo mật

Trong các hệ thống như PGP hoặc S/MIME:

- RSA được dùng để ký email
- Đảm bảo email không bị chỉnh sửa
- Xác minh đúng người gửi

c) Hóa đơn điện tử và chữ ký số tại Việt Nam

Trong các hệ thống khai thuế, hóa đơn điện tử:

- Doanh nghiệp sử dụng chữ ký số RSA để ký tài liệu
- Đảm bảo tính pháp lý và không thể chối bỏ

Quy trình ký hóa đơn điện tử tại Việt Nam

Bước 1: Chuẩn bị dữ liệu hóa đơn

- Doanh nghiệp tạo hóa đơn điện tử với đầy đủ thông tin: mã số thuế, tên khách hàng, số tiền, ngày phát hành…
- Hóa đơn được định dạng theo chuẩn XML hoặc PDF theo quy định của Tổng cục Thuế.

Bước 2: Tạo giá trị băm (Hash Value)

- Hệ thống sử dụng thuật toán băm (thường là SHA-256) để tạo ra chuỗi đại diện cho toàn bộ nội dung hóa đơn.
- Chuỗi băm này đảm bảo rằng nếu có bất kỳ thay đổi nào trong hóa đơn, giá trị băm sẽ khác ngay lập tức.

Bước 3: Ký số bằng RSA

- Chuỗi băm được mã hóa bằng khóa bí mật RSA của doanh nghiệp.
- Kết quả là chữ ký số RSA, gắn liền với hóa đơn điện tử.

Bước 4: Gửi hóa đơn đã ký

- Hóa đơn điện tử kèm chữ ký số được gửi đến cơ quan thuế và khách hàng.

- Người nhận có thể kiểm tra chữ ký bằng khóa công khai RSA của doanh nghiệp.

Bước 5: Kiểm tra và xác thực

- Cơ quan thuế/khách hàng tính lại giá trị băm từ nội dung hóa đơn.
- Dùng khóa công khai RSA để giải mã chữ ký số và thu được giá trị băm gốc.
- So sánh hai giá trị băm:
- Nếu trùng khớp → hóa đơn hợp lệ, không bị chỉnh sửa.
- Nếu khác nhau → hóa đơn không hợp lệ hoặc đã bị thay đổi.

d) Blockchain và tiền mã hóa

RSA (hoặc các biến thể tương tự) được sử dụng để:

- Xác thực giao dịch
- Đảm bảo chỉ chủ sở hữu mới có quyền sử dụng tài sản

e) Phần mềm và cập nhật hệ thống

Các nhà phát triển phần mềm sử dụng chữ ký số để:

- Ký file cài đặt
- Đảm bảo phần mềm không bị chỉnh sửa bởi hacker

Ví dụ: Windows, Android đều kiểm tra chữ ký trước khi cài đặt ứng dụng.

3.5. Cài đặt giải thuật sử dụng Python

https://github.com/qqtrung/RSA

- digital_signature_demo.py

```python
# ========================= HÀM HASH VÀ CHỮ KÝ =========================
def compute_hash(message: str, algorithm_name: str) -> Tuple[str, int]:
    """
    Tính hash của thông điệp theo thuật toán chỉ định.
    Trả về: (hash dưới dạng hex, hash dưới dạng số nguyên)
    """
    digest = hashlib.new(algorithm_name, message.encode("utf-8")).digest()
    return digest.hex(), int.from_bytes(digest, "big")


def sign_message(message: str, algorithm_label: str, key_pair: RSAKeyPair) -> SignatureData:
    """
    Tạo chữ ký số RSA cho thông điệp.
    Quy trình: Hash → mod n → ký bằng private key (mũ d)
    """
    algorithm_name = HASH_ALGORITHMS[algorithm_label]
    _, raw_hash_int = compute_hash(message, algorithm_name)

    reduced_hash = raw_hash_int % key_pair.n
    signature_int = pow(reduced_hash, key_pair.d, key_pair.n)   # Chữ ký = hash^d mod n

    return SignatureData(
        algorithm_label=algorithm_label,
        algorithm_name=algorithm_name,
        message=message,
        raw_hash_int=raw_hash_int,
        reduced_hash=reduced_hash,
        signature_int=signature_int,
    )
```

```python
def verify_message(
    message: str,
    signature_int: int,
    algorithm_label: str,
    n: int,
    e: int,
) -> VerificationData:
    """
    Kiểm tra tính hợp lệ của chữ ký số.
    Quy trình: Hash thông điệp → mod n → giải mã chữ ký bằng public key (mũ e)
    So sánh hai giá trị để quyết định chữ ký có hợp lệ không.
    """
    algorithm_name = HASH_ALGORITHMS[algorithm_label]
    _, raw_hash_int = compute_hash(message, algorithm_name)

    reduced_hash = raw_hash_int % n
    decrypted_signature = pow(signature_int, e, n)          # Giải mã chữ ký = signature^e mod n

    return VerificationData(
        algorithm_label=algorithm_label,
        message=message,
        signature_int=signature_int,
        raw_hash_int=raw_hash_int,
        reduced_hash=reduced_hash,
        decrypted_signature=decrypted_signature,
        is_valid=(reduced_hash == decrypted_signature),
    )
```

```python
# ========================= MÃ HÓA XOR + BASE64 =========================
def xor_cipher_base64(text: str, password: str, encrypt: bool = True) -> str:
    """
    Mã hóa / Giải mã thông điệp bằng phép XOR với mật khẩu,
    sau đó bọc thêm Base64 để tránh làm hỏng giao diện Tkinter.

    encrypt=True  → Mã hóa
    encrypt=False → Giải mã
    """
    if not password or not text:
        return text

    if encrypt:
        # Mã hóa XOR
        result = []
        for i, char in enumerate(text):
            pwd_char = password[i % len(password)]
            result.append(chr(ord(char) ^ ord(pwd_char)))

        # Bọc Base64 để chuỗi an toàn khi hiển thị trong Text widget
        return base64.b64encode("".join(result).encode('utf-8')).decode('utf-8')
    else:
        # Giải mã
        try:
            decoded = base64.b64decode(text.encode('utf-8')).decode('utf-8')
            result = []
            for i, char in enumerate(decoded):
                pwd_char = password[i % len(password)]
                result.append(chr(ord(char) ^ ord(pwd_char)))
            return "".join(result)
        except Exception:
            return "[Lỗi: Không thể giải mã XOR, dữ liệu bị hỏng hoặc không đúng định dạng]"

```

- shared_secret_rsa_demo.py

```python
    def generate_keys(self):
        """Tạo cặp khóa RSA mới cho cả bên A và bên B"""
        # Tạo khóa cho Bên A
        p = self.generate_prime(16)
        q = self.generate_prime(16)
        while q == p:
            q = self.generate_prime(16)
        self.key_A = self.build_key_pair(p, q, 65537)

        # Tạo khóa cho Bên B
        p = self.generate_prime(16)
        q = self.generate_prime(16)
        while q == p:
            q = self.generate_prime(16)
        self.key_B = self.build_key_pair(p, q, 65537)

        # Cập nhật hiển thị
        self.key_a_info.config(text=f"Khóa A → Public: (e={self.key_A.e}, n={self.key_A.n})")
        self.key_b_info.config(text=f"Khóa B → Public: (e={self.key_B.e}, n={self.key_B.n})")

        # Reset mật khẩu thu được
        self.secret_received_A = None
        self.secret_received_B = None
        self.secret_a_label.config(text="Chưa có dữ liệu")
        self.secret_b_label.config(text="Chưa có dữ liệu")

        self.status_var.set("Đã tạo khóa mới cho A và B. Hãy gửi số bí mật cho nhau.")

```

```python
    def send_secret(self, from_side: str):
        """Bên gửi số bí mật đến bên kia qua mã hóa RSA"""
        try:
            secret = int(self.secret_entry.get().strip())
        except ValueError:
            messagebox.showerror("Lỗi", "Vui lòng nhập một số nguyên hợp lệ")
            return

        if from_side == "A":
            if not self.key_A or not self.key_B:
                messagebox.showerror("Lỗi", "Chưa có khóa")
                return
            # A mã hóa bằng public key của B
            encrypted = pow(secret, self.key_B.e, self.key_B.n)
            decrypted = pow(encrypted, self.key_B.d, self.key_B.n)   # B giải mã

            self.b_result.config(state="normal")
            self.b_result.delete("1.0", "end")
            self.b_result.insert("1.0",
                f"Nhận từ A (đã mã hóa): {encrypted}\n\n"
                f"Giải mã được: {decrypted}\n"
                f"→ Bên B đã nhận được số bí mật từ A")
            self.b_result.config(state="disabled")

            self.secret_received_B = decrypted

        else:  # B gửi cho A
            if not self.key_A or not self.key_B:
                messagebox.showerror("Lỗi", "Chưa có khóa")
                return
            # B mã hóa bằng public key của A
            encrypted = pow(secret, self.key_A.e, self.key_A.n)
            decrypted = pow(encrypted, self.key_A.d, self.key_A.n)   # A giải mã

            self.a_result.config(state="normal")
            self.a_result.delete("1.0", "end")
            self.a_result.insert("1.0",
                f"Nhận từ B (đã mã hóa): {encrypted}\n\n"
                f"Giải mã được: {decrypted}\n"
                f"→ Bên A đã nhận được số bí mật từ B")
            self.a_result.config(state="disabled")

            self.secret_received_A = decrypted

        self.status_var.set(f"Bên {from_side} đã gửi số bí mật thành công.")
```

```python
    def create_shared_secret(self):
        """Tạo và hiển thị mật khẩu chung cho cả hai bên"""
        if not self.key_A or not self.key_B:
            messagebox.showwarning("Chưa có khóa", "Hãy tạo khóa trước khi tạo mật khẩu chung")
            return

        if self.secret_received_A is None or self.secret_received_B is None:
            messagebox.showwarning("Chưa đủ dữ liệu",
                                   "Cả hai bên cần gửi số bí mật cho nhau trước khi tạo mật khẩu chung.")
            return

        # Minh họa cách tạo shared secret (có thể cải tiến sau)
        # Ở đây dùng công thức đơn giản để cả hai bên ra cùng một kết quả
        self.shared_secret = (self.secret_received_A * self.secret_received_B) % (self.key_A.n * self.key_B.n)

        # Hiển thị riêng cho từng bên
        self.secret_a_label.config(
            text=f"{self.shared_secret}\n"
                 f"(Bên A tính từ số bí mật nhận được từ B)"
        )
        self.secret_b_label.config(
            text=f"{self.shared_secret}\n"
                 f"(Bên B tính từ số bí mật nhận được từ A)"
        )

        self.status_var.set("Mật khẩu chung đã được tạo thành công cho cả hai bên!")

        messagebox.showinfo("Thành công",
            f"Mật khẩu chung đã được tạo!\n\n"
            f"Giá trị: {self.shared_secret}\n\n"
            "Cả hai bên đều tính ra được cùng một mật khẩu mà không cần gửi trực tiếp.")
```

4. Kết luận

Chữ ký số là công cụ rất quan trọng trong giao dịch điện tử vì giúp:

- Xác thực người gửi.
- Đảm bảo tính toàn vẹn của dữ liệu.
- Chống chối bỏ trách nhiệm.
- Tăng giá trị pháp lý cho tài liệu điện tử.

RSA là thuật toán phổ biến nhất trong chữ ký số nhờ độ an toàn cao và dễ triển khai.

Tuy nhiên RSA vẫn có nhược điểm là tốc độ chậm và chi phí tính toán cao. Vì vậy trong thực tế RSA thường được kết hợp với các thuật toán băm mạnh như SHA-256 và các hệ mã hóa đối xứng như AES để đạt hiệu quả tốt hơn.

5. Tài liệu tham khảo

1. Hoàng Xuân Dậu (2015), *Giáo trình An toàn bảo mật hệ thống thông tin*, Nhà xuất bản Thông tin và Truyền thông, Học viện Công nghệ Bưu chính Viễn thông. (Trọng tâm: Chương 4 – Các kỹ thuật mã hóa thông tin).
2. Nguyễn Bình (2007), *Cơ sở lý thuyết mật mã*, Nhà xuất bản Học viện Kỹ thuật Mật mã.
3. Tiêu chuẩn Quốc gia TCVN 11816-3:2017, *Công nghệ thông tin – Các kỹ thuật an toàn – Thuật toán xác thực thực thể – Phần 3: Các cơ chế sử dụng kỹ thuật chữ ký số*.
4. William Stallings (2017), *Cryptography and Network Security: Principles and Practice* (7th Edition), Pearson Education. (Đây là tài liệu tham khảo chuẩn quốc tế về thuật toán RSA và chữ ký số).
5. NIST (2013), *FIPS PUB 186-4: Digital Signature Standard (DSS)*, National Institute of Standards and Technology. (Tiêu chuẩn của Viện Tiêu chuẩn và Công nghệ Quốc gia Mỹ về các thuật toán chữ ký số, bao gồm RSA).
6. Rivest, R. L., Shamir, A., & Adleman, L. (1978), "A Method for Obtaining Digital Signatures and Public-Key Cryptosystems", *Communications of the ACM*, Vol. 21, No. 2, pp. 120-126. (Bài báo gốc khai sinh ra thuật toán RSA).
7. Wikipedia, "RSA (cryptosystem)", [Trực tuyến]. Địa chỉ: https://en.wikipedia.org/wiki/RSA_(cryptosystem (Truy cập ngày: 13/04/2026).
8. RFC 8017, "PKCS #1: RSA Cryptography Specifications Version 2.2", Internet Engineering Task Force (IETF). [Trực tuyến]. Địa chỉ: https://tools.ietf.org/html/rfc8017.